MARMARA UNIVERSITY

FACULTY OF ENGINEERING

CSE3055-DATABASE SYSTEMS

(Fall 2025)

Step-IV

Due: 26.12.2025/23.59

Department	Student ID	Name & Surname
Computer Engineering	150123852	İlayda İlhan
Computer Engineering	150123035	Canan Pazarbaşı
Computer Engineering	150122050	Ömer Ercan Dayan

Table of Contents

1. PROJECT OVERVİEW & OBJECTIVE 3
2. TECHNICAL ARCHITECTURE 3
3. DATABASE CONNECTION & CONFIGURATION 3
3.1. Connection String Strategy 3
3.2. Dependency Injection & Service Registration 4
4. APPLICATION ENTRY & ROLE SEGREGATION 4
5. INSERT OPERATION: CUSTOMER REGISTRATION 5
6. CUSTOMER MODULE: AUTHENTICATION & DASHBOARD 9
6.1. Customer Login (Authentication) 9
6.3. User Dashboard & Session Management 10
6.4. Module A: Personal Info (View Profile) 11
6.5. Module B: Info Management (Update Profile) 12
7. STAFF MODULE: OPERATIONAL MANAGEMENT 15
7.1. Staff Authentication (Secure Entry) 16
7.2. Staff Dashboard (The Command Center) 16
7.3. The Staff Dashboard: Functional Overview 17
A) Customer Management Modules 17
B) Financial & Transactional Modules 19
C) Inventory Management Module 22
8. CONCLUSION 24

1. PROJECT OVERVİEW & OBJECTIVE

In this final phase of the **Dene Teknoloji Database Project**, a robust web-based interface has been developed. The primary objective is to abstract the complex underlying database structure (designed in Steps 1-3) into a user-friendly, secure, and interactive application.

This interface serves as the operational bridge, allowing both customers and employees to interact with the database—performing insertions, updates, deletions, and viewing reports—without requiring direct access to the SQL Server Management Studio.

2. TECHNICAL ARCHITECTURE

To ensure scalability, security, and maintainability, the application is built upon a modern software stack:

- **Backend Framework:** ASP.NET Core (Razor Pages)
- **Database:** Microsoft SQL Server
- **Design Pattern:** Repository Pattern (Separation of Data Access Layer and Business Logic)
- **Frontend:** Bootstrap 5 & CSS3 (For responsive and professional UI)
- **Security:** Role-Based Access Control (RBAC) separating Customer and Employee privileges.

3. DATABASE CONNECTION & CONFIGURATION

The foundation of the application is a secure and robust connection between the ASP.NET Core web server and the Microsoft SQL Server database. To ensure best practices, connection credentials are not hard-coded into the application logic but are managed through configuration files.

3.1. Connection String Strategy

The application utilizes the appsettings.json configuration file to store the database connection string. This approach allows for easy environment switching (Development vs. Production) without altering the source code.

```
Dashboard.cshtml   appsettings.json   CustomerRepository.cs   Index.cshtml.cs   Dashboard.cshtml.cs
Şema: https://www.schemastore.org/appsettings.json
1  {
2    "ConnectionStrings": {
3      "DeneTechnologyDB": "Server=localhost;Database=DENE;Integrated Security=True;TrustServerCertificate=True"
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "AllowedHosts": "*"
12 }
13
```

Figure 1. *Screenshot of the connection string code found in the appsetting.json file.*

3.2. Dependency Injection & Service Registration

In Program.cs, the database context and the Repository services are registered to the Dependency Injection (DI) container. This ensures that a new database connection is created and disposed of efficiently for each HTTP request, preventing memory leaks and connection exhaustion.

```
var connectionString = builder.Configuration.GetConnectionString("DeneTechnologyDB");

// 2. Repository Tanımlamaları
builder.Services.AddScoped<InvoiceRepository>(provider =>
    new InvoiceRepository(connectionString ?? throw new InvalidOperationException("Connection string not found")));

builder.Services.AddScoped<CustomerRepository>(provider =>
    new CustomerRepository(connectionString ?? throw new InvalidOperationException("Connection string not found")));

builder.Services.AddScoped<ProductRepository>(provider =>
    new ProductRepository(connectionString ?? throw new InvalidOperationException("Connection string not found")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();
```

Figure 2. *Repository definitions found in program.cs*

4. APPLICATION ENTRY & ROLE SEGREGATION

Upon launching the application, users are greeted by a central **Access Portal**. This page serves as the primary navigation hub, designed to enforce the separation of duties immediately.

User Interface Design: The portal uses a clean, card-based layout to clearly distinguish between the two primary actors in the system:

1. **Customer:** Redirects to the client-facing login and registration portal.
2. **Employee:** Redirects to the secured internal staff terminal.

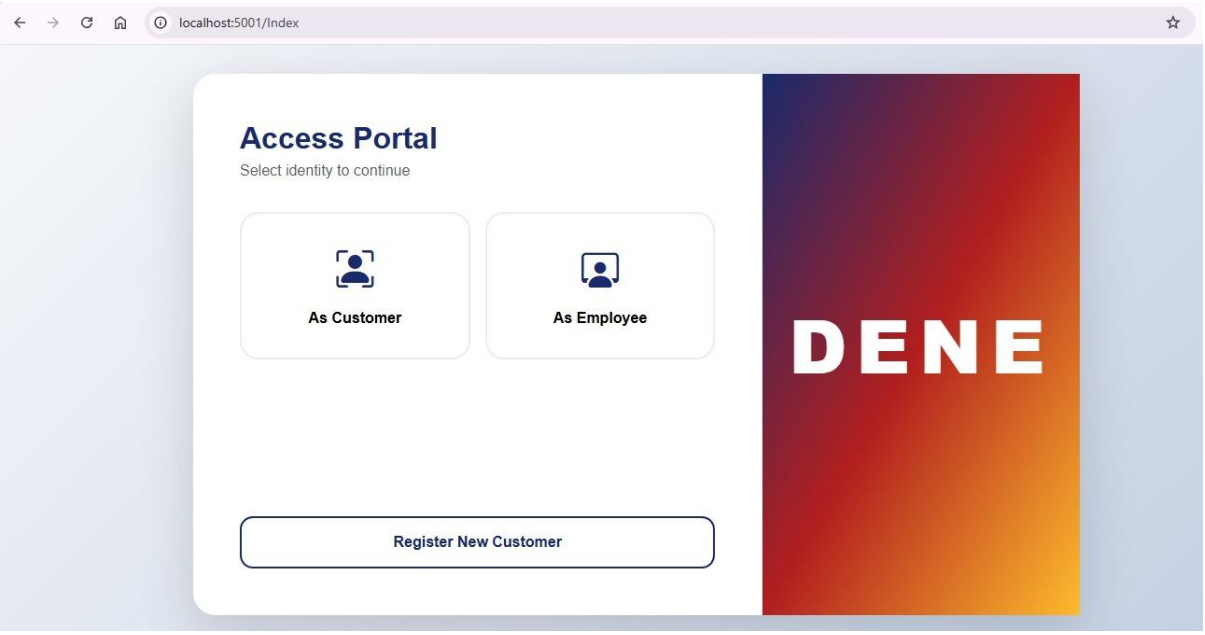


Figure 3. *The first screen the user encounters, where customer/employee selection or registration takes place.*

5. INSERT OPERATION: CUSTOMER REGISTRATION

Scenario: A new client wishes to interact with Dene Teknoloji. **Process:** The user selects "As Customer" from the portal and navigates to the "Register New Customer" page.

This interface represents the primary **INSERT** operation in the system. It collects user data (Name, Email, Phone, Address), validates the input, and commits the new record to the Customers table in the database.

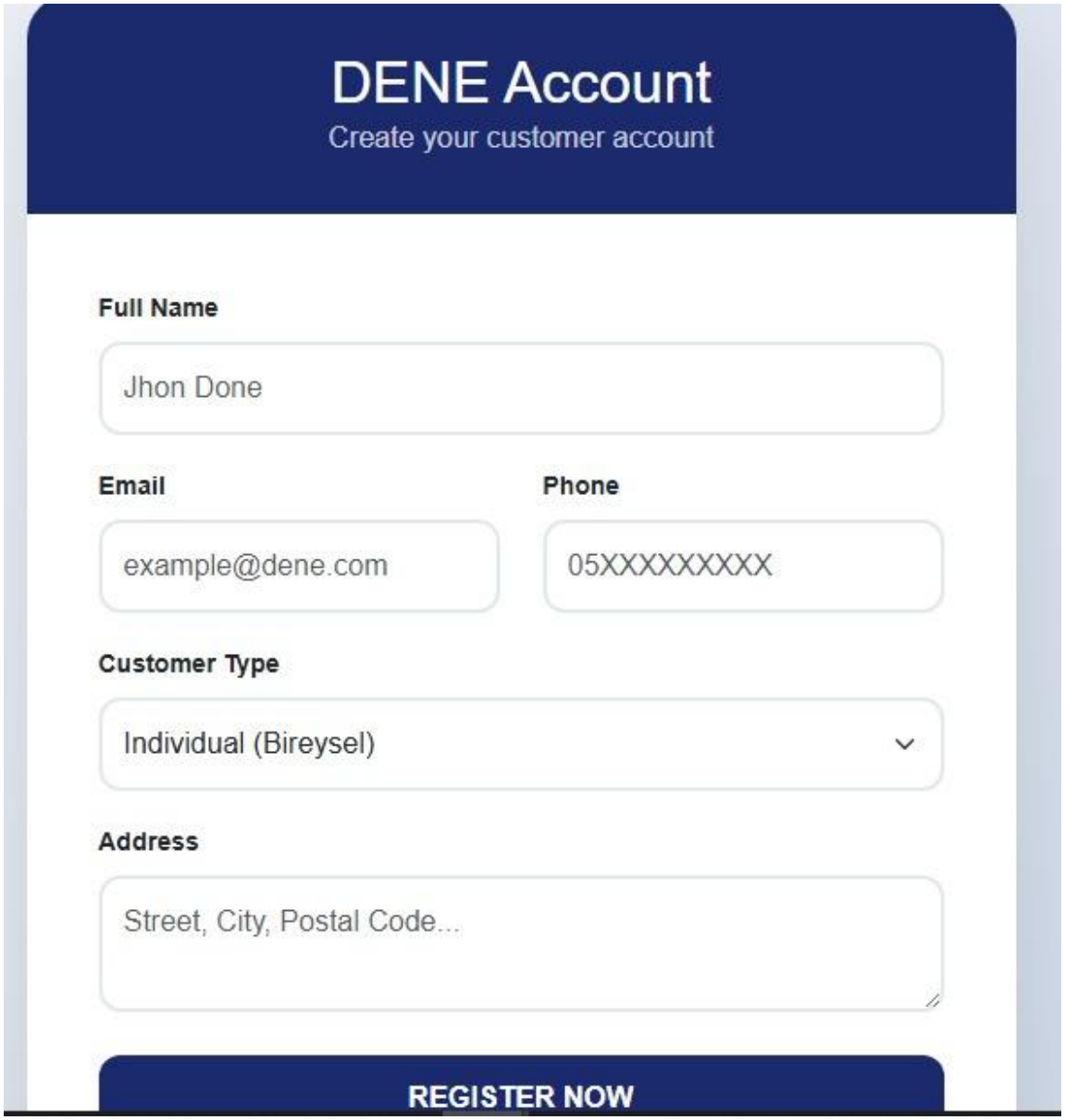


Figure 4. *This is the screen that appears after clicking the "Register for new customer" button.*

```
2 başvuru
public async Task<int> AddCustomer(Customer customer)
{
    using var connection = new SqlConnection(_connectionString);
    await connection.OpenAsync();
    using var command = new SqlCommand("SP_ADD_CUSTOMER", connection);
    command.CommandType = CommandType.StoredProcedure;
    command.Parameters.AddWithValue("@Name", customer.Name ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@Address", customer.Address ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@PhoneNumber", customer.Phone ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@Email", customer.Email ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@CustomerType", customer.CustomerType ?? (object)DBNull.Value);

    var outputId = new SqlParameter("@NewCustomerID", SqlDbType.Int) { Direction = ParameterDirection.Outpu
    command.Parameters.Add(outputId);
    await command.ExecuteNonQueryAsync();
    return (int)outputId.Value;
}
}
}
```

Figure 5. *The function used to add customers in CustomerRepository.cs*

DESKTOP-CUS...bo.Customer

CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
6	Burak Şahin	Konya	05008887766	burak.sahin@e...	Bireysel	2025-12-24
7	Zeynep Arslan	Eskişehir	05009990012	zeynep.arslan@...	Kurumsal	2025-12-24
8	Fatma Çelik	Kayseri	05006665544	fatma.celik@ex...	Bireysel	2025-12-24
9	Ahmet Öz	Gaziantep	05007774422	ahmet.oz@exa...	Kurumsal	2025-12-24
10	Selin Er	Mersin	05004561239	selin.er@exam...	Bireysel	2025-12-24
11	Hülya Kaplan	Tekirdağ	05009384756	hulya.kaplan@...	Kurumsal	2025-12-24
12	Orhan Bulut	Muğla	05002568974	orhan.bulut@e...	Bireysel	2025-12-24
13	Deniz Taş	Trabzon	05003847562	deniz.tas@exa...	Kurumsal	2025-12-24
14	Gamze Kar	Balıkesir	05007452319	gamze.kar@exa...	Bireysel	2025-12-24
15	Murat Uzun	Adana	05006123478	murat.uzun@e...	Kurumsal	2025-12-24
16	Aslı Tunç	Edirne	05003457812	asli.tunc@exa...	Bireysel	2025-12-24
17	Kerem Gök	Rize	05009234581	kerem.gok@ex...	Kurumsal	2025-12-24
18	Nazlı Kır	Aydın	05004321234	nazli.kir@exam...	Kurumsal	2025-12-24
19	Uğur Ateş	Van	05006549871	ugur.ates@exa...	Bireysel	2025-12-24
20	Sevgi Baran	Malatya	05007891234	sevgi.baran@ex...	Kurumsal	2025-12-24
21	Tolga Cenk	Kütahya	05005678123	tolga.cenk@ex...	Bireysel	2025-12-24
22	Melis Gür	Sakarya	05009998877	melis.gur@exa...	Bireysel	2025-12-24
23	Okan Sert	Denizli	05003458912	okan.sert@exa...	Kurumsal	2025-12-24
24	Yasemin Kaan	Hatay	05009341256	yasemin.kaan@...	Bireysel	2025-12-24
25	Efe Demirtaş	Samsun	05008461239	efe.demirtas@e...	Kurumsal	2025-12-24
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 6. *Database status before a new customer is added.*

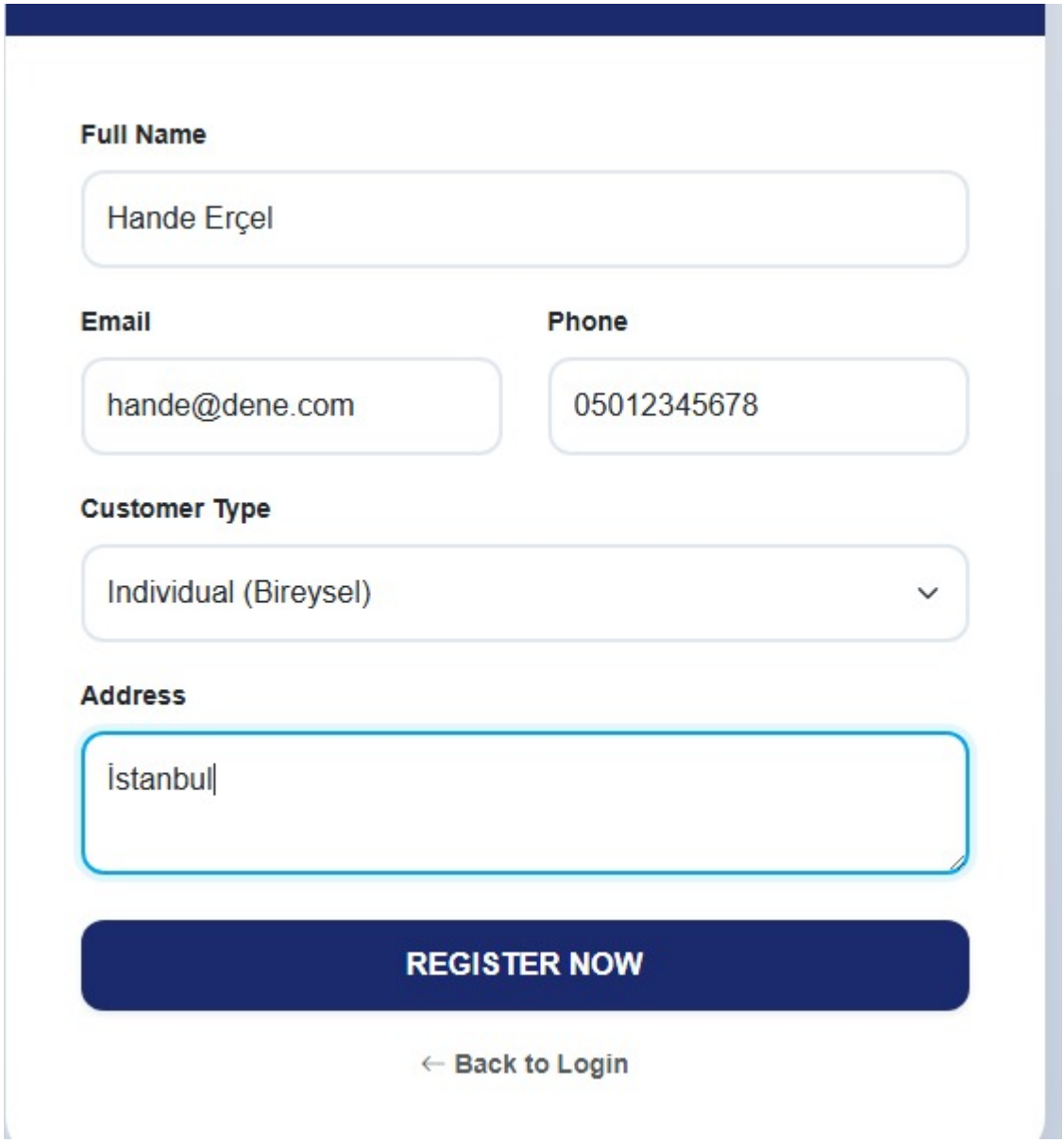


Figure 7. *Information entered during registration*

18	Nazlı Kır	Aydın	05004321234	nazli.kir@exam...	Kurumsal	2025-12-24
19	Uğur Ateş	Van	05006549871	ugur.ates@exa...	Bireysel	2025-12-24
20	Sevgi Baran	Malatya	05007891234	sevgi.baran@ex...	Kurumsal	2025-12-24
21	Tolga Cenk	Kütahya	05005678123	tolga.cenk@ex...	Bireysel	2025-12-24
22	Melis Gür	Sakarya	05009998877	melis.gur@exa...	Bireysel	2025-12-24
23	Okan Sert	Denizli	05003458912	okan.sert@exa...	Kurumsal	2025-12-24
24	Yasemin Kaan	Hatay	05009341256	yasemin.kaan@...	Bireysel	2025-12-24
25	Efe Demirtaş	Samsun	05008461239	efe.demirtas@e...	Kurumsal	2025-12-24
28	Hande Erçel	İstanbul	05012345678	hande@dene.c...	Individual	2025-12-25
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 8. *This is the result of being automatically added to the customer table in the database after registration.*

6. CUSTOMER MODULE: AUTHENTICATION & DASHBOARD

Upon selecting "As Customer" from the Access Portal, the user enters the customer-specific workflow. This module focuses on secure access and personalized data presentation.

6.1. Customer Login (Authentication)

Scenario: An existing customer wants to access their account. **Process:** The user enters their credentials (Name/ID). The backend validates these credentials against the Customers table.

This operation represents a critical **SELECT** query where the system searches for a matching record. If found, a session is initiated.

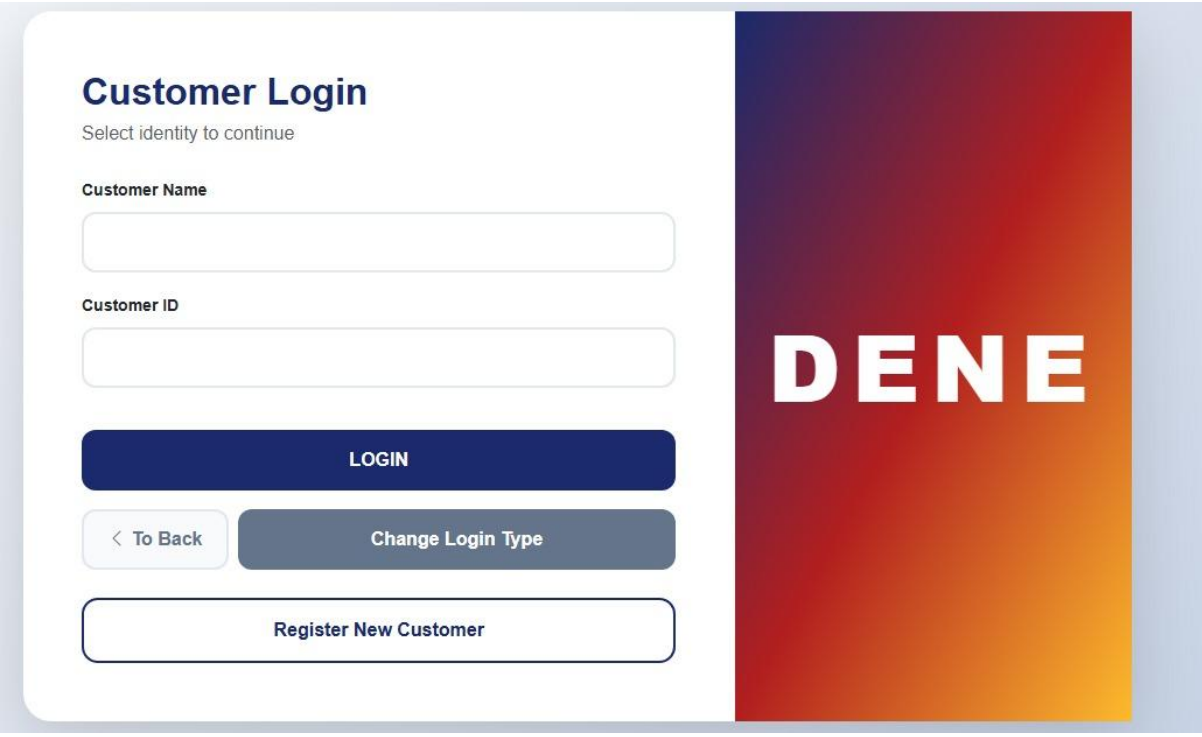


Figure 9. *Customer login screen*

Here, a real name from the database is used as the customer's name (example: Ali Yılmaz), and the password is 123.

```
// SELECT: ID'ye göre müşteri getir
5 başvuru
public async Task<Customer?> GetCustomerById(int customerId)
{
    using var connection = new SqlConnection(_connectionString);
    await connection.OpenAsync();
    // image_ff3161.png hatasını çözmek için gerekli olan adres ve tip alanları eklendi
    using var command = new SqlCommand("SELECT CustomerID, Name, Email, PhoneNumber, Address, CustomerType
    command.Parameters.AddWithValue("@CustomerID", customerId);
    using var reader = await command.ExecuteReaderAsync();
    if (await reader.ReadAsync())
    {
        return new Customer
        {
            CustomerID = reader.GetInt32(0),
            Name = reader.IsDBNull(1) ? null : reader.GetString(1),
            Email = reader.IsDBNull(2) ? null : reader.GetString(2),
            Phone = reader.IsDBNull(3) ? null : reader.GetString(3),
            Address = reader.IsDBNull(4) ? null : reader.GetString(4),
            CustomerType = reader.IsDBNull(5) ? null : reader.GetString(5)
        };
    }
    return null;
}
```

Figure 10. *The code for the Select operation in CustomerRepository.cs is shown in the screenshot.*

6.3. User Dashboard & Session Management

Upon successful authentication, the user is directed to the **Customer Dashboard**. This interface is dynamically generated based on the active user session.

Data Binding Verification: As seen in the visual below, the system greets the user by name (**"Welcome back, Ali Yılmaz"**). This text is not static; it is fetched directly from the Customers table upon login, proving that the session management and database retrieval are functioning correctly.

The dashboard presents two primary functional modules:

1. **Personal Info (Read Operation):** For viewing registered details.
2. **Info Management (Update Operation):** For editing account settings and security preferences.

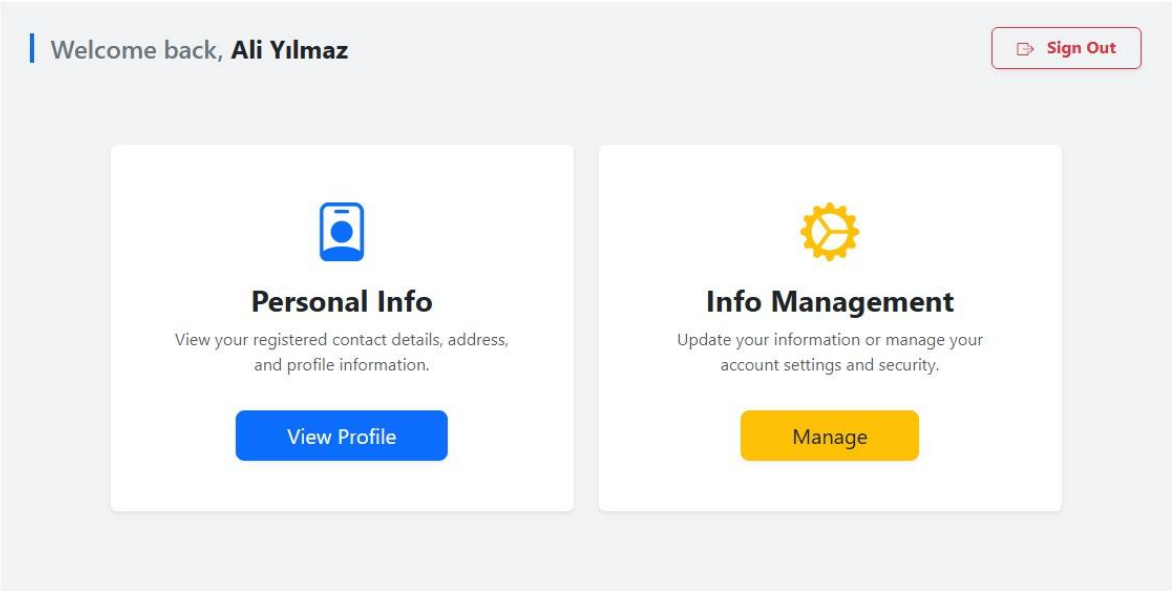


Figure 11. *The screen a customer encounters after logging into the system.*

6.4. Module A: Personal Info (View Profile)

Scenario: The user wants to verify their contact details or address.

Database Operation: SELECT (Read)

When the user clicks the **"View Profile"** button, the system triggers a query to fetch the complete profile of the logged-in user. This ensures that the customer always sees the most up-to-date information stored in the database.

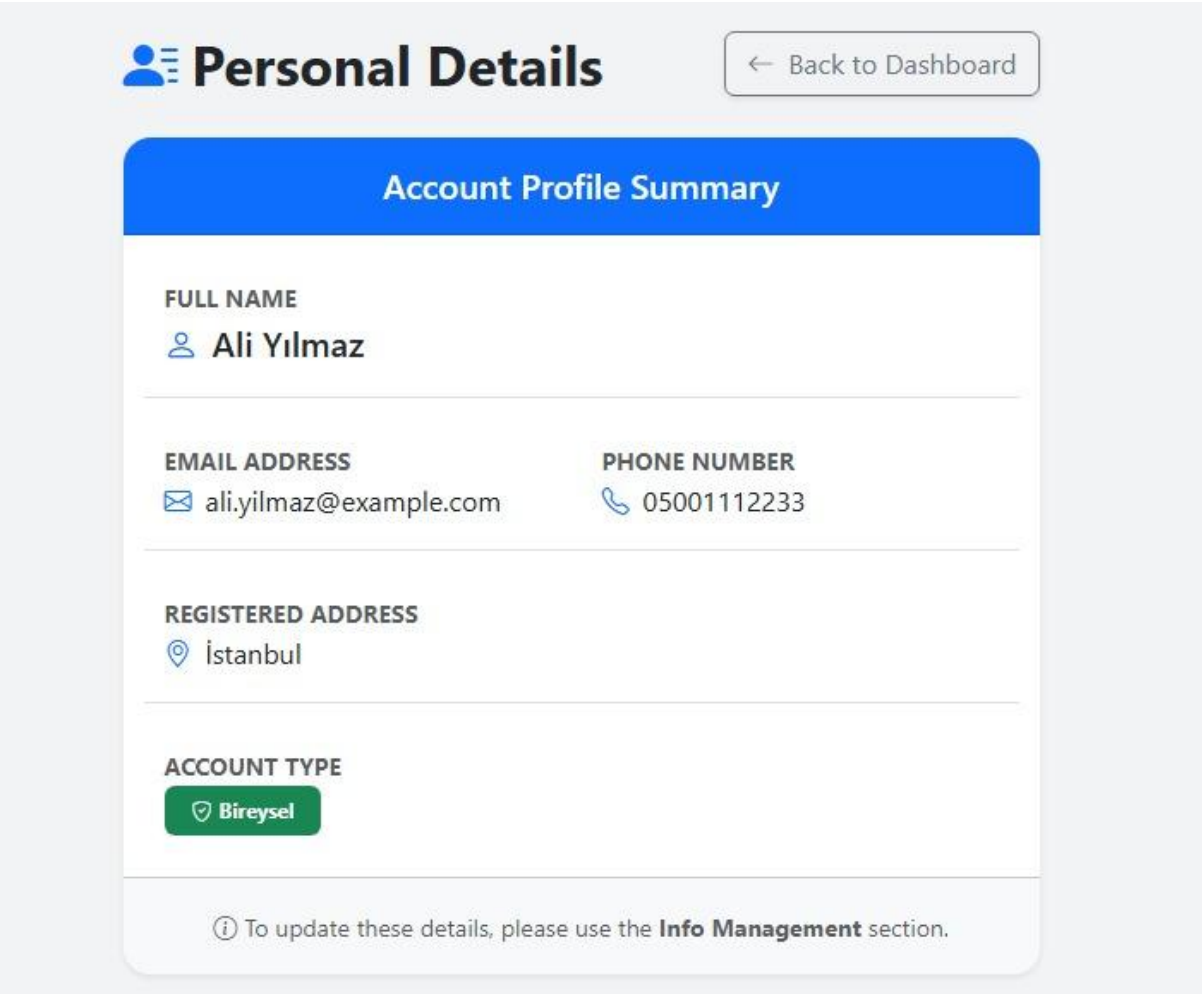


Figure 12. *The screen that opens after clicking the personal details button.*

6.5. Module B: Info Management (Update Profile)

Scenario: The user needs to update their phone number or change their password. **Database Operation:** UPDATE

The **"Manage"** button redirects the user to the editing form. Here, the system utilizes the UPDATE command (or a stored procedure like sp_CustomerUpdate) to modify existing records. This demonstrates the application's ability to handle data consistency and modification requests securely.

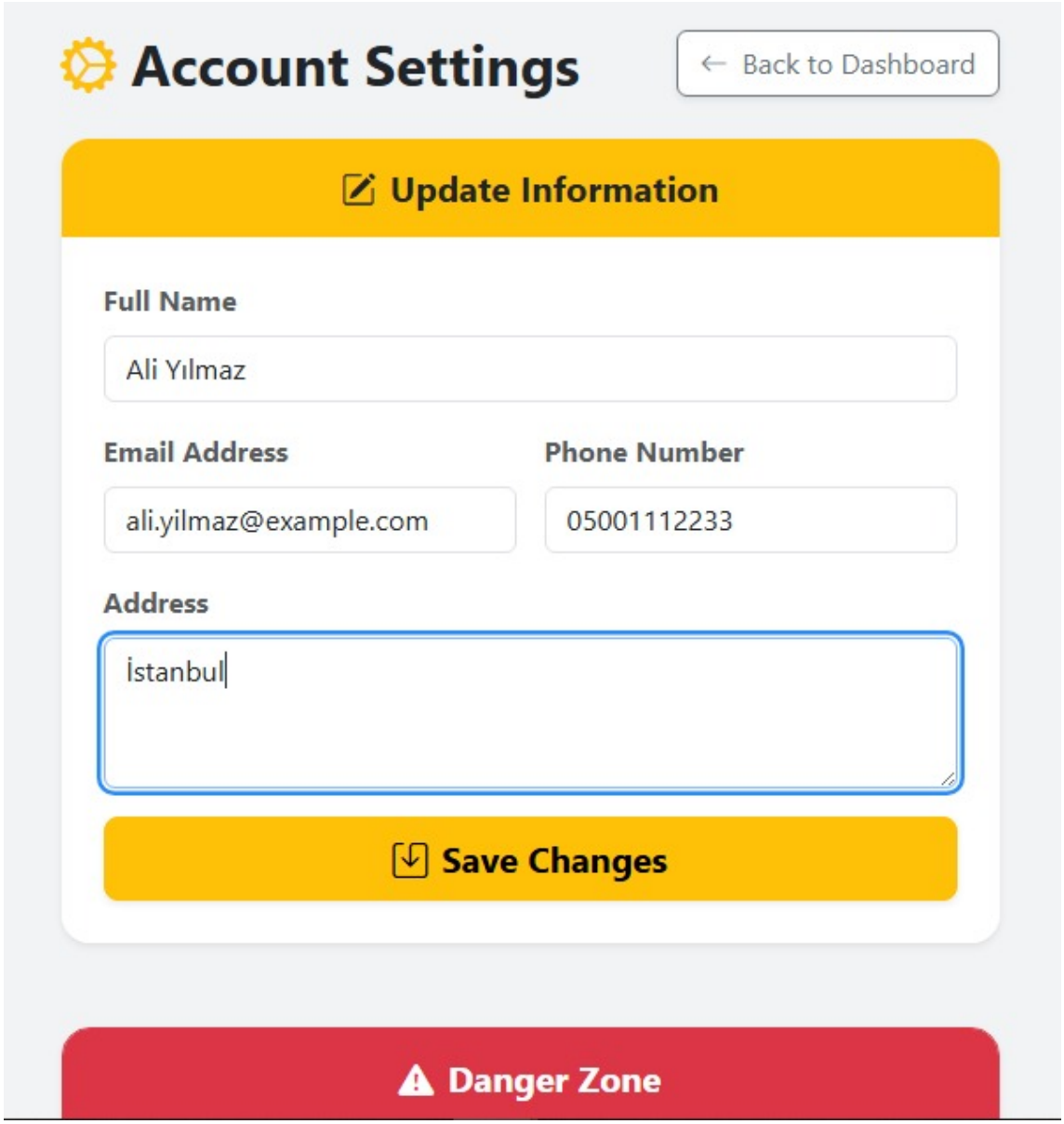


Figure 13. *The screen that appears after selecting the update information button.*

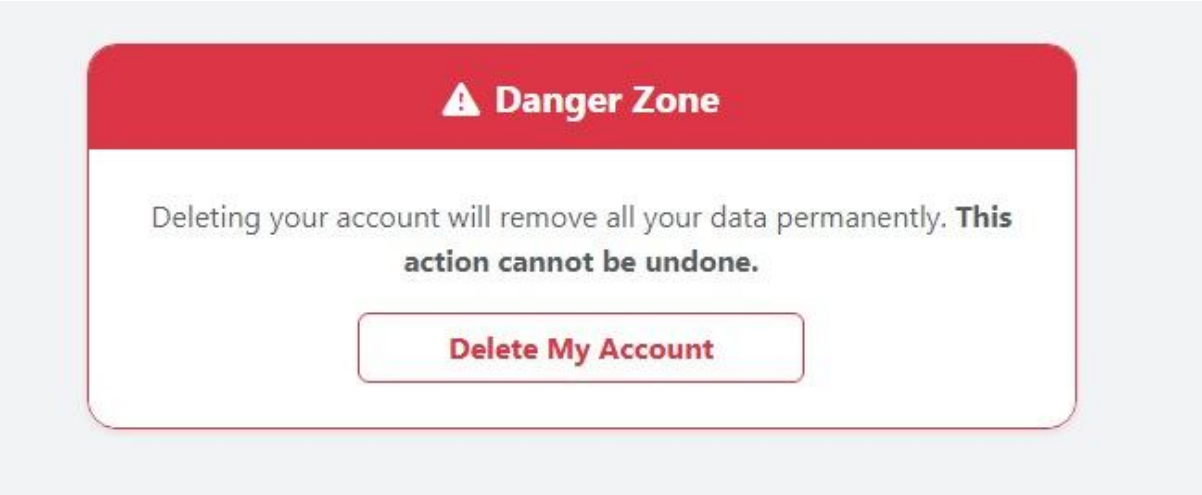


Figure 14. *Warning message found in the update information tab.*

CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
1	Ali Yılmaz	İstanbul	05001112233	ali.yilmaz@exa...	Bireysel	2025-12-24
2	Ayşe Demir	Ankara	05003334455	ayse.demir@ex...	Kurumsal	2025-12-24

Figure 15. *The state of the customer table in the database before the data was updated.*

```
2 başvuru
public async Task<bool> UpdateCustomer(int oldId, int newId, Customer customer, int totalInvoices, decimal
{
    using var connection = new SqlConnection(_connectionString);
    await connection.OpenAsync();
    using var command = new SqlCommand("SP_UPDATE_CUSTOMER", connection);
    command.CommandType = CommandType.StoredProcedure;

    command.Parameters.AddWithValue("@OldCustomerID", oldId);
    command.Parameters.AddWithValue("@NewCustomerID", newId);
    command.Parameters.AddWithValue("@Name", customer.Name ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@Address", customer.Address ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@PhoneNumber", customer.Phone ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@Email", customer.Email ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@CustomerType", customer.CustomerType ?? (object)DBNull.Value);
    command.Parameters.AddWithValue("@TotalInvoices", totalInvoices);
    command.Parameters.AddWithValue("@TotalSpent", totalSpent);

    using var reader = await command.ExecuteReaderAsync();
    if (await reader.ReadAsync())
    {
        return reader.GetInt32(0) == 1; // SQL'den dönen ResultCode
    }
    return false;
}
```

Figure 16. *Code related to the update process found in CustomerRepository.cs*

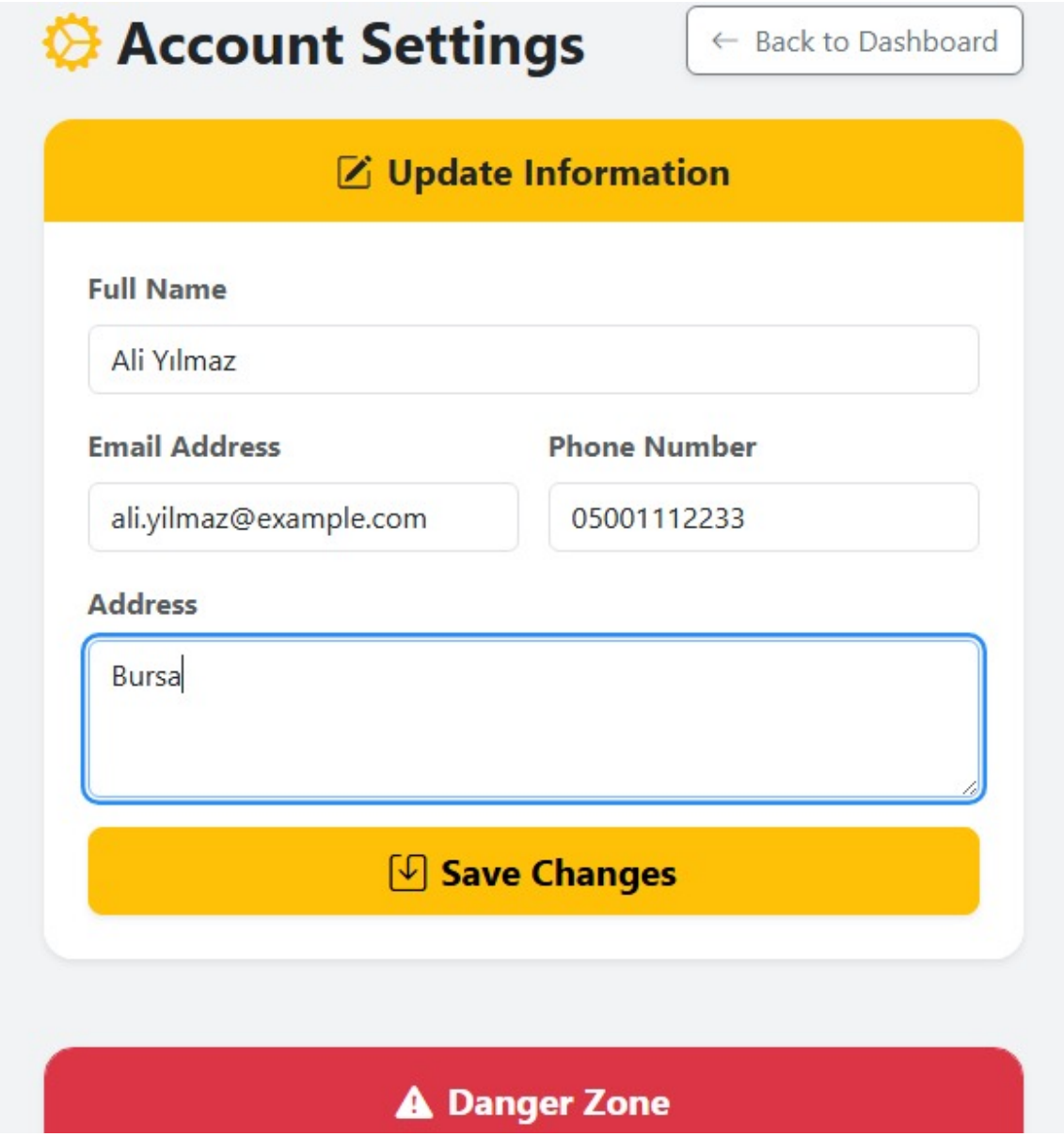


Figure 17. *Ali Yılmaz's address information is changed to Bursa.*

	CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
▶	1	Ali Yılmaz	Bursa	05001112233	ali.yilmaz@exa...	*NULL*	2025-12-24
	2	Ayşe Demir	Ankara	05003334455	ayse.demir@ex...	Kurumsal	2025-12-24
	3	Mehmet Kaya	İzmir	05001234567	mehmet.kaya@...	Bireysel	2025-12-24

Figure 18. *The updated address information for Ali Yılmaz in the database after the change.*

7. STAFF MODULE: OPERATIONAL MANAGEMENT

While the Customer Module focuses on personal data management, the **Staff Module** is designed for high-level database operations. This section demonstrates the implementation of complex business rules, such as stock management and invoicing, which require elevated privileges.

7.1. Staff Authentication (Secure Entry)

Scenario: An authorized employee accesses the internal system.

Security Protocol: The system enforces a strict separation between customer and employee tables. When a user attempts to log in via the Staff Portal, the backend queries the Employees table exclusively.

Employees enter "admin" as both their employee name and password when logging in.

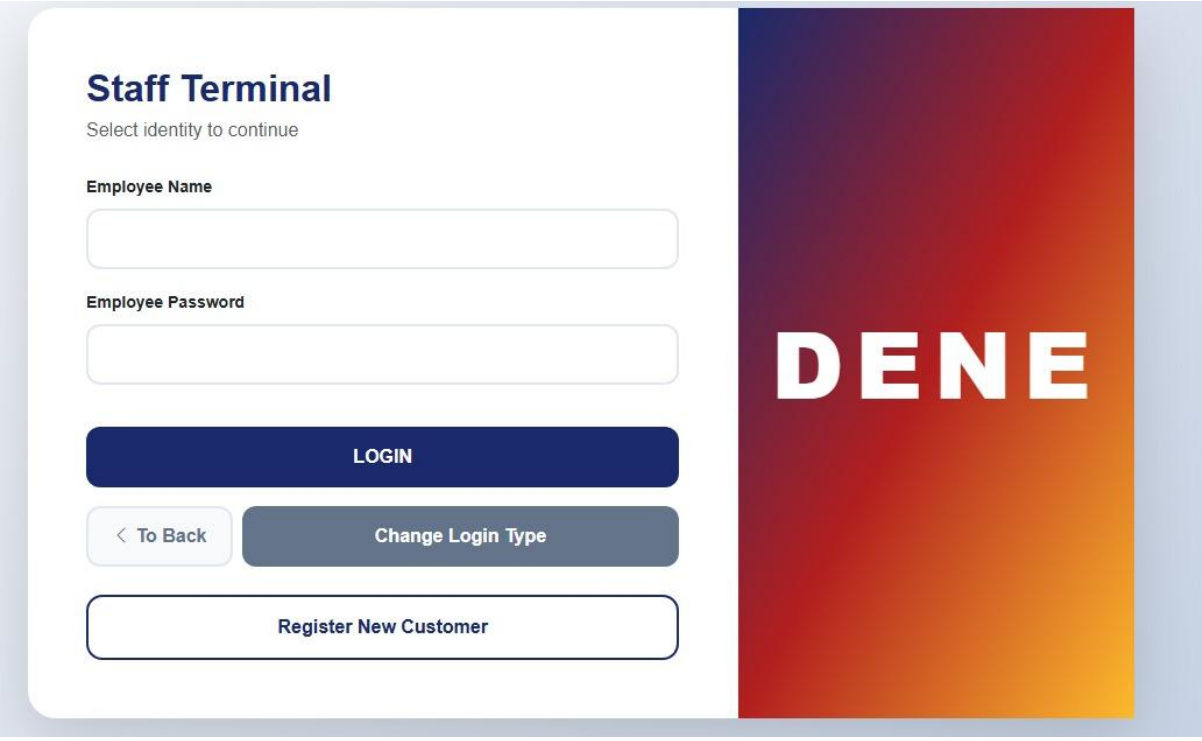


Figure 19. *Login screen for employees*

7.2. Staff Dashboard (The Command Center)

Scenario: Operational Overview. **Design Philosophy:** The dashboard is designed as a "Command Center," providing direct access to all critical CRUD operations via a card-based layout.

Functional Map:

- **New Customer:** Quick access to the registration flow (Insert).
- **Customer Report:** Advanced filtering and listing of client data (Select/Reporting).
- **New Invoice:** Transaction processing (Insert into multiple tables: Invoice & InvoiceDetails).
- **Delete Invoice:** Administrative correction (Delete / Soft Delete).
- **Update Stock:** Inventory management (Update via Stored Procedure).

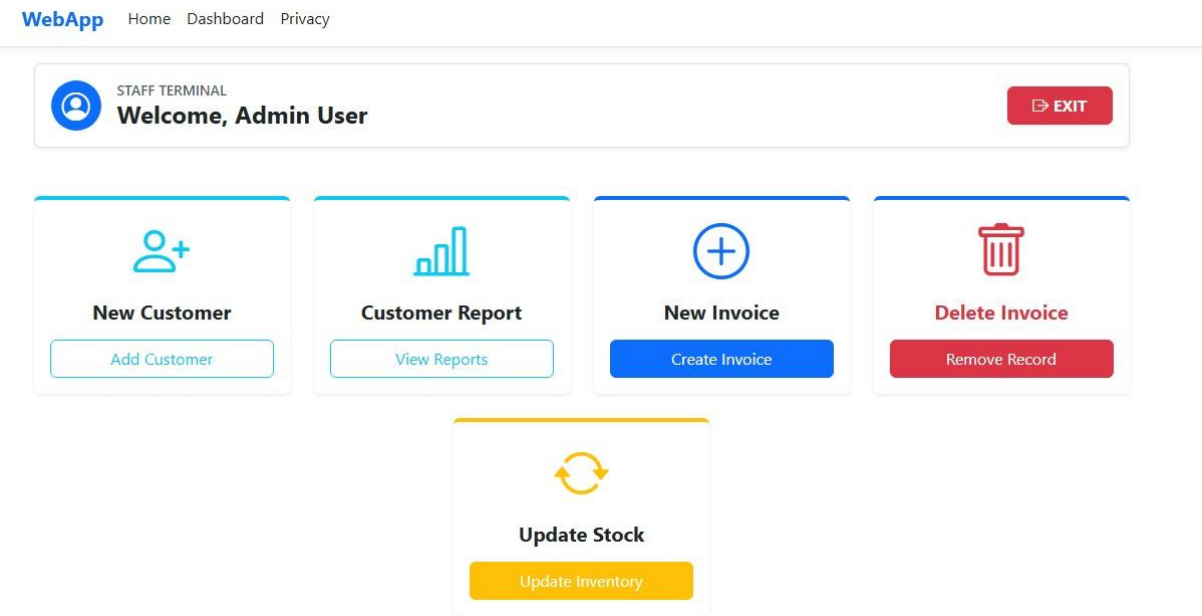


Figure 20. *Screen that opens after logging in*

7.3. The Staff Dashboard: Functional Overview

Upon successful login, the operational staff is presented with the Staff Dashboard. This interface is designed to provide immediate access to the system's core capabilities without navigating through complex menus.The dashboard is divided into 5 Functional Modules, each corresponding to a specific database operation:

A) Customer Management Modules

- **1. New Customer (INSERT):**
  - *Function:* Redirects to the registration form.
  - *Purpose:* Allows staff to manually register a client who might be present physically or calling via phone.
  - *Backend:* Reuses the CustomerRepository.Add() method.

	22	Melis Gür	Sakarya	05009998877	melis.gur@exa...	Bireysel	2025-12-24
	23	Okan Sert	Denizli	05003458912	okan.sert@exa...	Kurumsal	2025-12-24
	24	Yasemin Kaan	Hatay	05009341256	yasemin.kaan@...	Bireysel	2025-12-24
	25	Efe Demirtaş	Samsun	05008461239	efe.demirtas@e...	Kurumsal	2025-12-24
	28	Hande Erçel	İstanbul	05012345678	hande@dene.c...	Individual	2025-12-25
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 21. *The Customer table in the database before a new customer was added.*

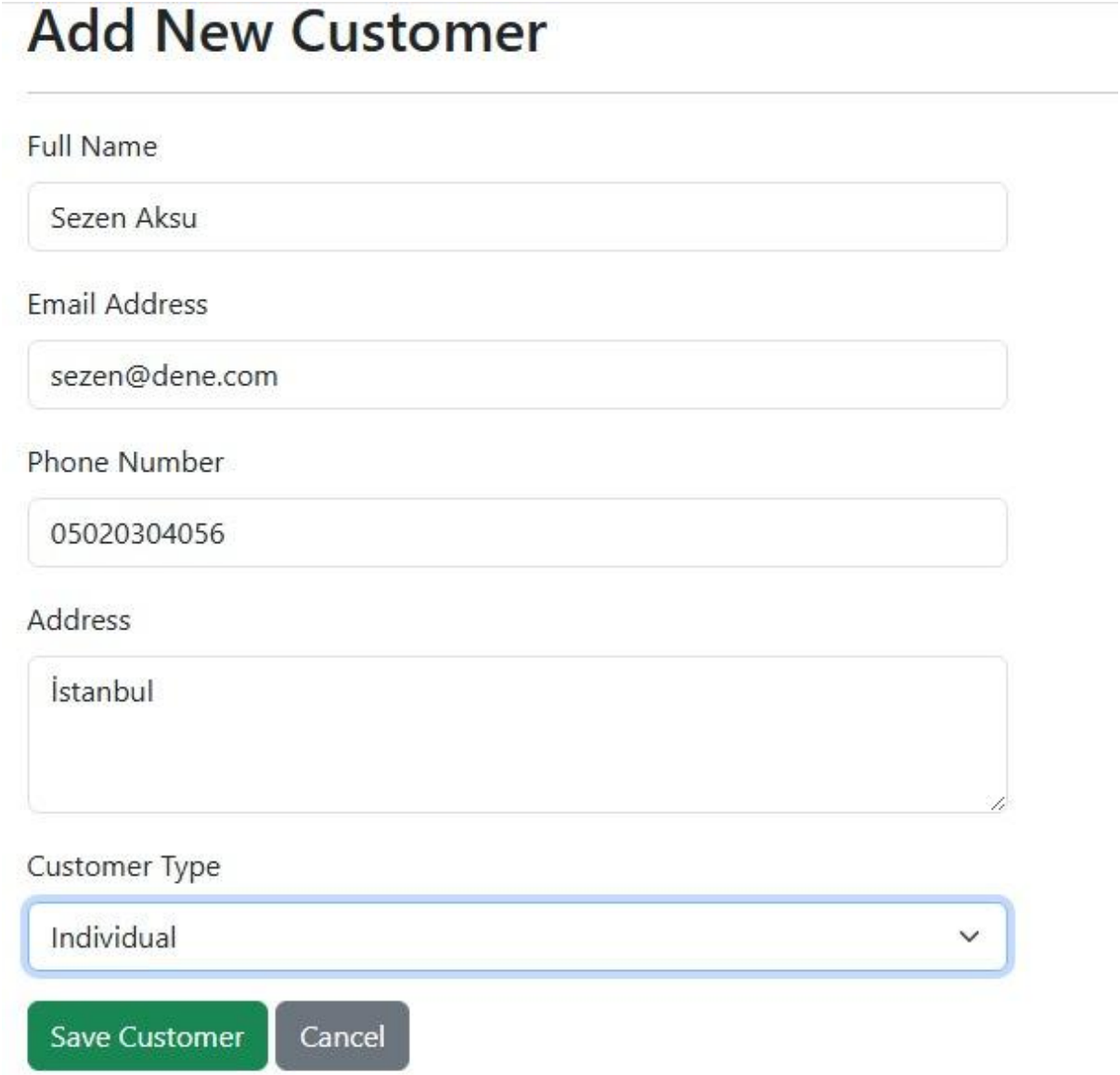


Figure 22. *A new customer adding screen on the web side.*

	28	Hande Erçel	İstanbul	05012345678	hande@dene.c...	Individual	2025-12-25
	29	Sezen Aksu	İstanbul	05020304056	sezen@dene.co...	Individual	2025-12-25
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 23. *The Customer table in the database after a new customer was added.*

2. Customer Report (SELECT):

- *Function:* Generates a list of all active clients.
- *Purpose:* Used for reporting and data verification.
- *Backend:* Triggers a SELECT * FROM Customers query (or a View) to populate a data grid.

Information updated successfully!

ID	Customer Name	Invoices	Total Spent	Actions
1	Ali Yılmaz	2	$7.020,00	Update Delete
12	Orhan Bulut	1	$5.074,00	Update Delete
9	Ahmet Öz	1	$4.320,00	Update Delete
20	Sevgi Baran	1	$3.420,00	Update Delete
15	Murat Uzun	1	$2.950,00	Update Delete
7	Zeynep Arslan	1	$2.640,00	Update Delete
23	Okan Sert	1	$2.478,00	Update Delete
17	Kerem Gök	1	$2.376,00	Update Delete
14	Gamze Kar	1	$2.040,00	Update Delete

Figure 24. *Related customer report screen.*

B) Financial & Transactional Modules

- **3. New Invoice (Complex INSERT):**
  - *Function:* Initiates the invoicing process.
  - *Purpose:* This is a transactional operation that affects multiple tables (Invoices, InvoiceDetails) and triggers stock calculations.
  - *Backend:* Utilizes a transaction block to ensure data integrity.

DESKTOP-CUSP... dbo.Invoice | DESKTOP-CUS...dbo.Customer

InvoiceID	InvoiceDate	CustomerID	ProductID	TotalAmount	TaxRate	TaxOfficeNum...	InvoiceType	InvoicingParty	InvoicedPa
7	2023-02-04	7	7	2200,00	20,00	ESK-FRM-02	Sales	Dene Teknoloji	Zeynep Ars
8	2023-02-09	8	8	980,00	18,00	KYS-DMR-10	Sales	Dene Teknoloji	Fatma Çelik
9	2023-02-12	9	9	3600,00	20,00	GZT-TBR-21	Sales	Dene Teknoloji	Ahmet Öz
10	2023-02-18	10	10	740,00	20,00	MRS-ERL-33	Return	Selin Er	Dene Teknc
11	2023-03-03	11	11	650,00	20,00	TKR-KPL-44	Sales	Dene Teknoloji	Hülya Kapl
12	2023-03-07	12	12	4300,00	18,00	MUG-BLT-22	Sales	Dene Teknoloji	Orhan Bulu
13	2023-03-09	13	13	1200,00	20,00	TRB-DNZ-12	Sales	Dene Teknoloji	Deniz Taş
14	2023-03-15	14	14	1700,00	20,00	BLK-KAR-56	Sales	Dene Teknoloji	Gamze Kar
15	2023-03-20	15	15	2500,00	18,00	ADN-UZN-78	Sales	Dene Teknoloji	Murat Uzur
16	2023-04-02	16	16	880,00	20,00	EDR-TNC-03	Return	Aslı Tunç	Dene Teknc
17	2023-04-06	17	17	1980,00	20,00	RİZ-GOK-19	Sales	Dene Teknoloji	Kerem Gök
18	2023-04-10	18	18	640,00	20,00	AYD-KIR-88	Sales	Dene Teknoloji	Nazlı Kır
19	2023-04-15	19	19	1100,00	20,00	VAN-ATS-04	Sales	Dene Teknoloji	Uğur Ateş
20	2023-04-19	20	20	2850,00	20,00	MLT-BRN-90	Sales	Dene Teknoloji	Sevgi Barar
21	2023-05-03	21	21	720,00	20,00	KUT-CNK-11	Return	Tolga Cenk	Dene Teknc
22	2023-05-08	22	22	450,00	20,00	SAK-GUR-14	Sales	Dene Teknoloji	Melis Gür
23	2023-05-12	23	23	2100,00	18,00	DNZ-SRT-66	Sales	Dene Teknoloji	Okan Sert
24	2023-05-18	24	24	980,00	20,00	HTY-KAN-77	Return	Yasemin Kaan	Dene Teknc
25	2023-05-22	25	25	1600,00	20,00	SMS-DMR-55	Sales	Dene Teknoloji	Efe Demirta
26	2025-12-24	1	1	5000,00	20,00	VD-Test	Sales	Dene	Customer
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 25. *Before performing the Insert Invoice operation, the invoice table in the database*

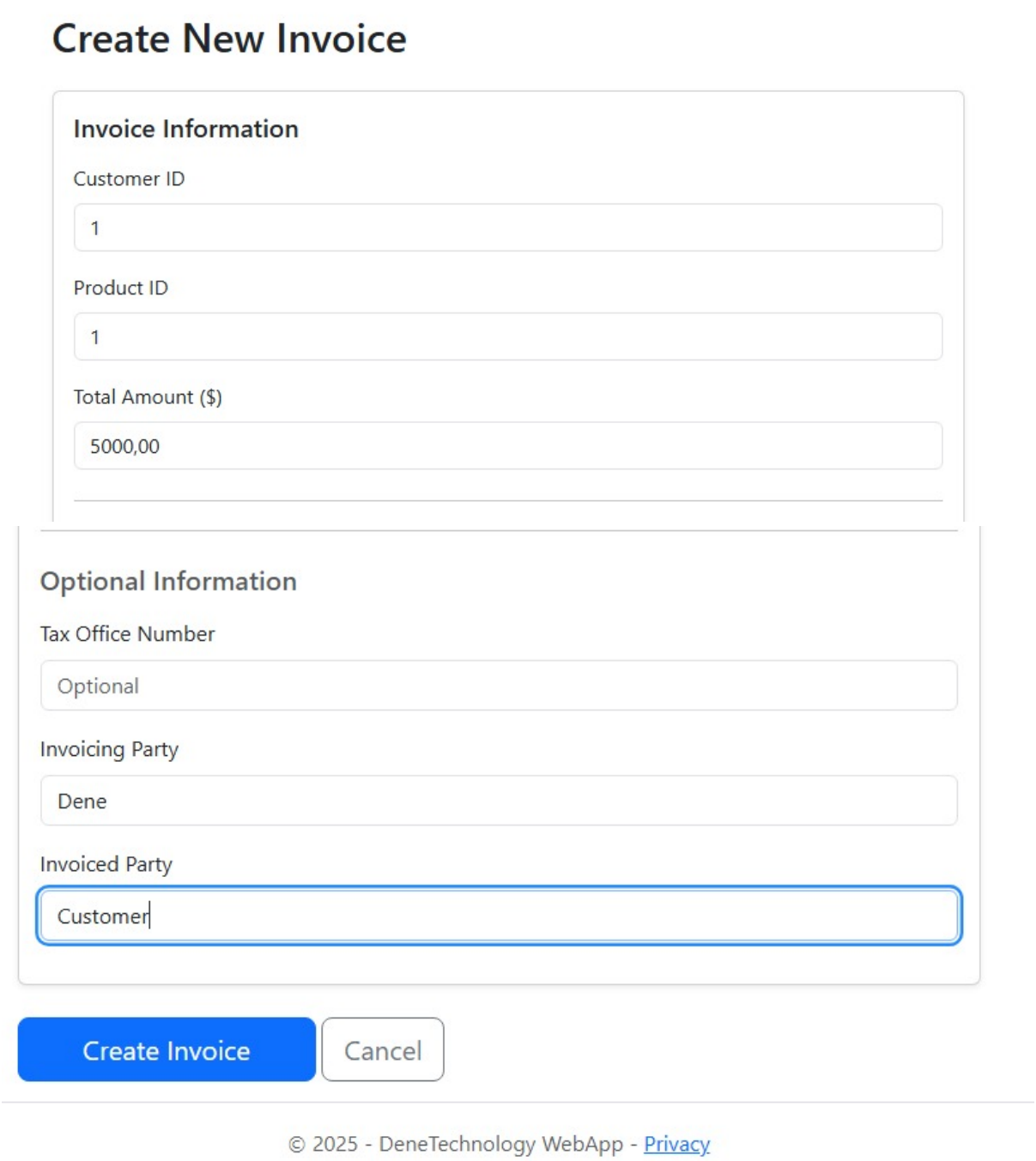


Figure 26&27. *The screen showing the Insert Invoice operation on the web.*

25	2023-05-22	25	25	1600,00	20,00	SMS-DMR-55	Sales	Dene Teknoloji	Efe Demirtaş	1920,000000000
26	2025-12-24	1	1	5000,00	20,00	VD-Test	Sales	Dene	Customer	6000,000000000
27	2025-12-25	1	1	5000,00	20,00	NULL	Sales	Dene	Customer	6000,000000000
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 28. *After performing the Insert Invoice operation, the invoice table in the database*

4. Delete Invoice (DELETE):

- *Function:* Removes an erroneous record.
- *Purpose:* Administrative capability to correct mistakes.
- *Backend:* Executes a DELETE command (or sets IsDeleted = 1 for soft delete) based on the Invoice ID.

	25	2023-05-22	25	25	1600,00	20,00	SMS-DMR-55	Sales	Dene Teknoloji	Efe Demirtaş	1920,000000000
	26	2025-12-24	1	1	5000,00	20,00	VD-Test	Sales	Dene	Customer	6000,000000000
	27	2025-12-25	1	1	5000,00	20,00	NULL	Sales	Dene	Customer	6000,000000000
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 29. *The Invoice table in the database before the deletion process.*

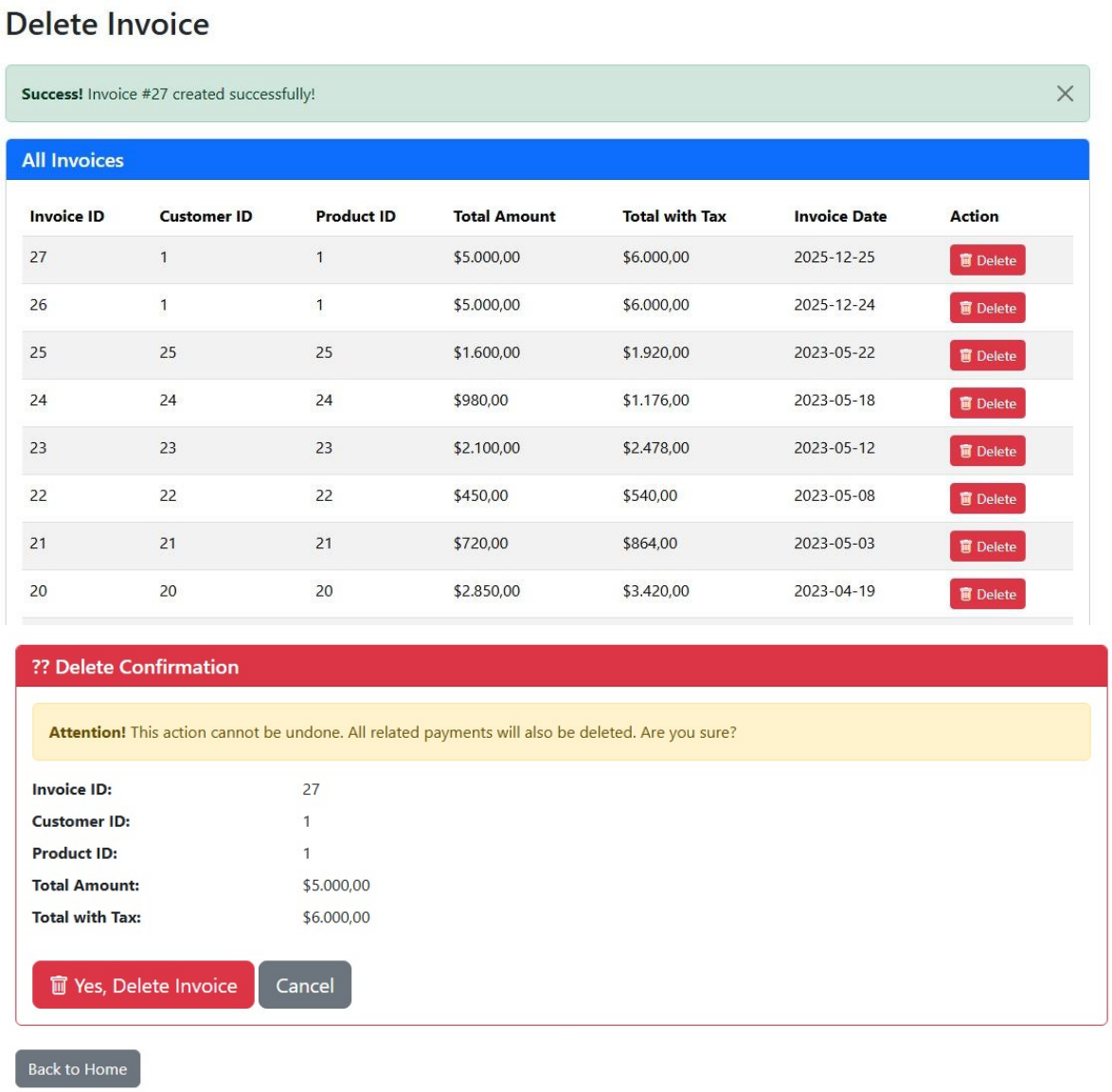
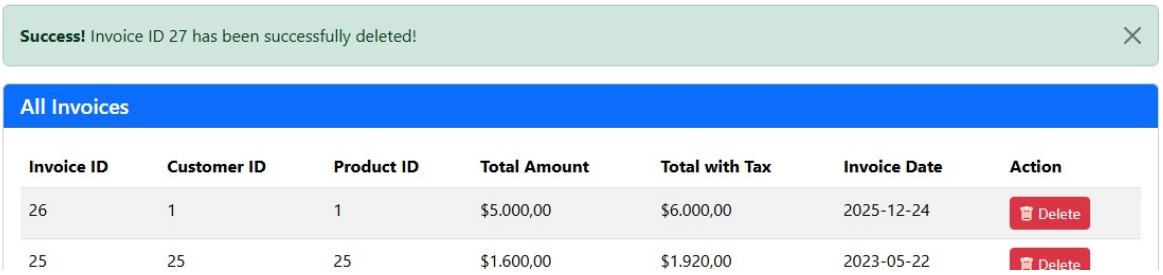


Figure 30&31. *The screen that opens on the web when the delete option is selected.*



Success! Invoice ID 27 has been successfully deleted!

All Invoices

Invoice ID	Customer ID	Product ID	Total Amount	Total with Tax	Invoice Date	Action
26	1	1	$5.000,00	$6.000,00	2025-12-24	Delete
25	25	25	$1.600,00	$1.920,00	2023-05-22	Delete

Figure 32. *Deletion of the term achievement message from the Ministry of National Education.*

11	2023-03-03	11	11	650,00	20,00	TKR-KPL-44	Sales	Dene Teknoloji	Hülya Kaplan	780,000000
12	2023-03-07	12	12	4300,00	18,00	MUG-BLT-22	Sales	Dene Teknoloji	Orhan Bulut	5074,00000
13	2023-03-09	13	13	1200,00	20,00	TRB-DNZ-12	Sales	Dene Teknoloji	Deniz Taş	1440,00000
14	2023-03-15	14	14	1700,00	20,00	BLK-KAR-56	Sales	Dene Teknoloji	Gamze Kar	2040,00000
15	2023-03-20	15	15	2500,00	18,00	ADN-UZN-78	Sales	Dene Teknoloji	Murat Uzun	2950,00000
16	2023-04-02	16	16	880,00	20,00	EDR-TNC-03	Return	Aslı Tunç	Dene Teknoloji	1056,00000
17	2023-04-06	17	17	1980,00	20,00	RİZ-GOK-19	Sales	Dene Teknoloji	Kerem Gök	2376,00000
18	2023-04-10	18	18	640,00	20,00	AYD-KIR-88	Sales	Dene Teknoloji	Nazlı Kır	768,000000
19	2023-04-15	19	19	1100,00	20,00	VAN-ATS-04	Sales	Dene Teknoloji	Uğur Ateş	1320,00000
20	2023-04-19	20	20	2850,00	20,00	MLT-BRN-90	Sales	Dene Teknoloji	Sevgi Baran	3420,00000
21	2023-05-03	21	21	720,00	20,00	KUT-CNK-11	Return	Tolga Cenk	Dene Teknoloji	864,000000
22	2023-05-08	22	22	450,00	20,00	SAK-GUR-14	Sales	Dene Teknoloji	Melis Gür	540,000000
23	2023-05-12	23	23	2100,00	18,00	DNZ-SRT-66	Sales	Dene Teknoloji	Okan Sert	2478,00000
24	2023-05-18	24	24	980,00	20,00	HTY-KAN-77	Return	Yasemin Kaan	Dene Teknoloji	1176,00000
25	2023-05-22	25	25	1600,00	20,00	SMS-DMR-55	Sales	Dene Teknoloji	Efe Demirtaş	1920,00000
26	2025-12-24	1	1	5000,00	20,00	VD-Test	Sales	Dene	Customer	6000,00000
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 33. *The Invoice table in the database after the deletion process.*

C) Inventory Management Module

- **5. Update Stock (UPDATE):**
  - *Function:* Adjusts product quantities.
  - *Purpose:* Allows the warehouse manager to correct stock levels manually or add new inventory.
  - *Backend:* Calls a specialized Stored Procedure (e.g., sp_UpdateProductStock) to safely modify the Products table.

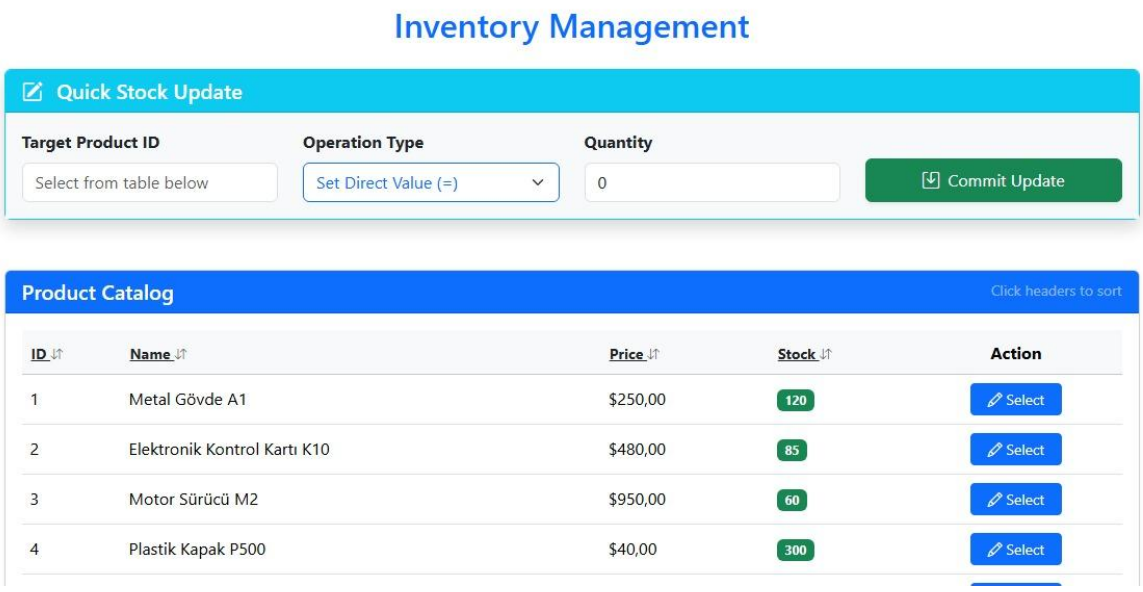


Figure 34. *Update screen*

DESKTOP-CUSP... dbo.Product | DESKTOP-CUSP...- dbo.Invoice | DESKTOP-CUS...dbo.Customer

ProductID	ProductName	UnitPrice	Category	Cost	Stock	Manufactured...	SupplierID	MachineID	QualityStatu
1	Metal Gövde A1	250,00	Metal Parça	180,00	120	2022-01-10	1	3	Pass
2	Elektronik Kont...	480,00	Elektronik	350,00	85	2021-03-15	2	4	Pass
3	Motor Sürücü ...	950,00	Motor	720,00	60	2020-07-22	19	5	NULL
4	Plastik Kapak P...	40,00	Plastik	15,00	300	2021-02-10	10	9	NULL
5	Hassas Sensör S...	780,00	Sensör	620,00	40	2023-04-01	23	25	NULL
6	Fiber Kablo F12	150,00	Kablo	90,00	500	2022-05-12	22	7	NULL
7	Servo Motor S...	1200,00	Motor	900,00	35	2020-11-05	12	8	NULL
8	Montaj Parçası ...	95,00	Montaj	55,00	200	2022-03-18	14	11	NULL
9	Robotik Kol Bile...	2100,00	Robotik	1650,00	25	2021-01-25	20	20	NULL
10	Kauçuk Yastık K...	65,00	Kauçuk	30,00	260	2023-02-01	5	14	NULL
11	Soğutma Ünite...	3300,00	Soğutma	2500,00	10	2020-09-21	13	12	NULL
12	Polimer Kasnak...	180,00	Polimer	70,00	140	2019-06-15	20	16	NULL
13	Hidrolik Pompa...	2700,00	Hidrolik	2100,00	18	2022-08-11	16	17	NULL
14	Montaj Aparatı ...	120,00	Montaj	65,00	180	2021-10-02	15	15	NULL
15	Servo Kontrol Ü...	680,00	Elektronik	450,00	55	2023-03-12	7	13	NULL
16	Optik Lens OL25	550,00	Optik	320,00	22	2020-12-05	25	18	NULL
17	Endüstriyel Fan ...	1500,00	Soğutma	1100,00	15	2021-07-30	6	6	NULL
18	Konveyör Zincir...	230,00	Konveyör	130,00	95	2022-09-01	7	2	NULL
19	Elektrik Bağlant...	55,00	Kablo	20,00	400	2023-01-09	4	10	NULL
20	Motor Vidası M...	12,00	Vida	2,00	800	2021-06-22	1	1	NULL
21	Boru Seti BS40	340,00	Metal Parça	220,00	140	2019-11-14	11	24	NULL
22	Plastik Panel PP...	90,00	Plastik	30,00	160	2020-08-17	10	22	NULL

1 / 26 | Hücre Salt Okunur.

Figure 35. *The product table in the database before the update*

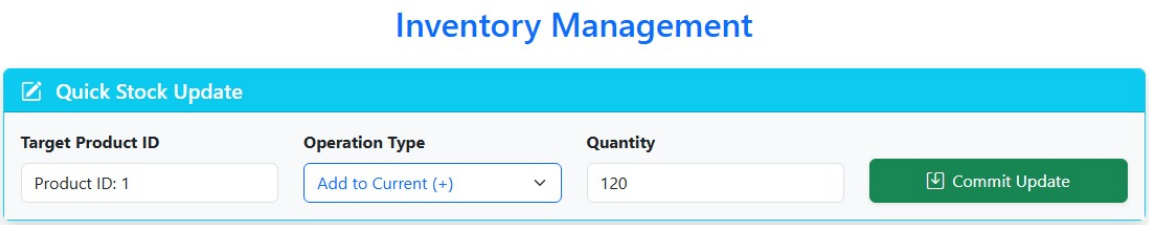


Figure 36. *Selected insertion operation*

DESKTOP-CUSP... dbo.Product | DESKTOP-CUSP...- dbo.Invoice | DESKTOP-CUS...dbo.Customer

ProductID	ProductName	UnitPrice	Category	Cost	Stock	Manufactured...	SupplierID	MachineID	QualityStatus
1	Metal Gövde A1	250,00	Metal Parça	180,00	240	2022-01-10	1	3	Pass
2	Elektronik Kont...	480,00	Elektronik	350,00	85	2021-03-15	2	4	Pass
3	Motor Sürücü ...	950,00	Motor	720,00	60	2020-07-22	19	5	NULL
4	Plastik Kapak P...	40,00	Plastik	15,00	300	2021-02-10	10	9	NULL
5	Hassas Sensör S...	780,00	Sensör	620,00	40	2023-04-01	23	25	NULL
6	Fiber Kablo F12	150,00	Kablo	90,00	500	2022-05-12	22	7	NULL
7	Servo Motor S...	1200,00	Motor	900,00	35	2020-11-05	12	8	NULL
8	Montaj Parçası ...	95,00	Montaj	55,00	200	2022-03-18	14	11	NULL
9	Robotik Kol Bile...	2100,00	Robotik	1650,00	25	2021-01-25	20	20	NULL
10	Kauçuk Yastık K...	65,00	Kauçuk	30,00	260	2023-02-01	5	14	NULL
11	Soğutma Ünite...	3300,00	Soğutma	2500,00	10	2020-09-21	13	12	NULL
12	Polimer Kasnak...	180,00	Polimer	70,00	140	2019-06-15	20	16	NULL
13	Hidrolik Pompa...	2700,00	Hidrolik	2100,00	18	2022-08-11	16	17	NULL
14	Montaj Aparatı ...	120,00	Montaj	65,00	180	2021-10-02	15	15	NULL
15	Servo Kontrol Ü...	680,00	Elektronik	450,00	55	2023-03-12	7	13	NULL
16	Optik Lens OL25	550,00	Optik	320,00	22	2020-12-05	25	18	NULL
17	Endüstriyel Fan ...	1500,00	Soğutma	1100,00	15	2021-07-30	6	6	NULL
18	Konveyör Zincir...	230,00	Konveyör	130,00	95	2022-09-01	7	2	NULL
19	Elektrik Bağlant...	55,00	Kablo	20,00	400	2023-01-09	4	10	NULL
20	Motor Vidası M...	12,00	Vida	2,00	800	2021-06-22	1	1	NULL
21	Boru Seti BS40	340,00	Metal Parça	220,00	140	2019-11-14	11	24	NULL
22	Plastik Panel PP...	90,00	Plastik	30,00	160	2020-08-17	10	22	NULL

Figure 37. *The product table in the database after the update*

8. CONCLUSION

In this final phase of the CSE3055 Database Systems course project, the conceptual database design for **Dene Teknoloji** has been successfully transformed into a fully operational web-based management system. By leveraging **ASP.NET Core** for the backend and **Microsoft SQL Server** for data storage, the project achieved a secure, scalable, and user-friendly environment.

Key technical and functional achievements include:

- **Architectural Integrity:** The implementation of the **Repository Design Pattern** ensured a clean separation of concerns between the data access layer and the business logic, making the code maintainable and testable.
- **Security & Access Control:** A strict Role-Based Access Control (RBAC) system was established, successfully segregating "Customer" data access from "Employee" operational privileges.
- **Data Consistency:** Critical business rules were enforced not just at the application level but also within the database using **Stored Procedures** and **Transactions**. This was particularly vital for complex operations such as *Invoicing* and *Stock Updates*, ensuring that partial failures do not corrupt the database state.
- **User Experience:** The web interface abstracts the complexity of raw SQL operations, providing an intuitive dashboard for users to perform CRUD operations efficiently.

Ultimately, this project demonstrates the practical application of database theory, resulting in a robust software solution that guarantees data security, integrity, and operational efficiency.